🟢 Điểm sáng (Strengths)

1. **Routing theo Role tốt**: Bạn đã thiết lập thư mục `dashboard/admin`, `dashboard/center`, `dashboard/expert`, `dashboard/parent`. Đây là mô hình Role-Based Routing chuẩn, giúp cô lập giao diện và logic riêng cho từng nhóm người dùng.
2. **Sử dụng Server Actions**: Bạn đã bắt kịp xu hướng khi dùng Server Actions (`"use server"`) cho các thao tác thay đổi dữ liệu (mutation) như trong `auth.ts`, `center.ts`.
3. **Cấu trúc Auth Firebase khá tốt**: Đã phân tách rõ ràng `admin.ts` (Firebase Admin SDK cho server side) và `config.ts` (Client Firebase). Chạy kiến trúc Cookie-based session cho bảo mật là một điểm cộng rất lớn (tốt hơn dùng localStorage).

🔴 Thiết kế mang nợ kỹ thuật (Technical Debt) cần xử lý ngay

WARNING

Những điểm này nếu không sửa sớm, khi team bạn có thêm người hoặc dự án phình to, code sẽ trở thành một "nồi lẩu thập cẩm" không thể maintain nổi (Spaghetti code).

1. Thiếu tư duy chia nhỏ Component (Componentization)

- **Thực trạng**: Thư mục `src/components` của bạn hiện tại gần như... trống rỗng (chỉ có đúng `ThemeProvider.tsx`). Toàn bộ UI phức tạp đang bị nhồi nhét chung vào các file Route. Điển hình nhất là file `src/app/dashboard/layout.tsx` dài tới > 310 dòng!
- **Hậu quả**: Khi bạn muốn dùng lại nút bấm, thẻ thông tin báo cáo, hay ô tìm kiếm ở chỗ khác, bạn sẽ phải copy-paste code. Lỗi giao diện thì cực kỳ khó tìm.
- **Tiêu chuẩn mở rộng**: Bạn cần kiến trúc "Thành phần UI độc lập". Tách `layout.tsx` ra thành nhiều file:
  - `src/components/layout/Sidebar.tsx`

- `src/components/layout/Header.tsx`
- `src/components/ui/SearchInput.tsx`, v.v. (Nên tham khảo cài đặt UI thư viện như Shadcn/ui).

2. Vi phạm nguyên lý "Separation of Concerns" (Tách biệt UI và Logic)

- **Thực trạng**: Khoảng dòng 30-65 trong `layout.tsx`, bạn đang gọi trực tiếp cơ sở dữ liệu (Firestore `collection`, `query`, `getDocs`) để làm tính năng Search ngay bên trong một giao diện Client Component!
- **Tiêu chuẩn công nghiệp**: UI không được biết cơ sở dữ liệu nằm ở đâu và truy vấn ra sao.
- **Cách khắc phục**:
  - Nên đẩy logic gọi database vào Server Action/Server Component.
  - Hoặc nếu bắt buộc dùng phía Client, hãy tạo một custom hook: `src/hooks/useSearchCenter.ts` để bọc (wrap) logic này lại, `layout.tsx` chỉ việc gọi hàm lấy dữ liệu sạch.

3. Cấu trúc thư mục App/Actions là Anti-Pattern ở Next.js

- **Thực trạng**: Bạn đặt các thư mục chứa logic server vào `src/app/actions`.
- **Tiêu chuẩn**: Thư mục `app/` (App Router) **chỉ nên** dùng cho việc định tuyến URL (chứa `page.tsx`, `layout.tsx`, `route.ts`). Các file logic business đặt trong này sẽ gây hiểu lầm cho người mới vào team.
- **Cách khắc phục**: Chuyển ngay thư mục `actions` ra ngoài, ngang cấp với `app`. Ví dụ: `src/actions/auth.ts` hoặc `src/lib/actions/auth.ts`.

4. Hardcode dữ liệu trong Code (Hardcoding Business Rules)

CAUTION

Đây là lỗi bảo mật và maintain cực kỳ nghiêm trọng ở môi trường DN.

- **Thực trạng**: Ở dòng 23 file `auth.ts`, bạn có đoạn: `if (email === "leduyvu27022005@gmail.com") { role = "admin"; }`. Bạn cố tình tạo backdoor cho một email cụ thể để nâng lên admin.

- **Tiêu chuẩn công nghiệp**: Mọi cấu hình về quyền hay môi trường KHÔNG được ghim chết trong mã nguồn. Nếu dev nghỉ việc, người đó vẫn giữ quyền Admin vĩnh viễn và bạn phải deploy lại toàn bộ dự án để xóa đoạn if đó đi.
- **Cách khắc phục**: Lưu danh sách này trong biến môi trường `.env` (`ADMIN_EMAILS=leduy...`) hoặc cấp quyền một lần qua script Firebase Admin, sau đó code chỉ việc đọc Custom Claim từ Token mà quyết định quyền hạ.

📝 Đề xuất quy hoạch lại thư mục (Architecture Blueprint)

Để sẵn sàng mở rộng, quy chuẩn hóa lại `src/` theo hướng Feature-Based Folder hoặc Modular:

```
plaintext
src/
 ├── actions/              # Chứa toàn bộ Server Actions (chu
yển từ src/app/actions sang)
 ├── app/                  # (Bảo toàn, chỉ chứa page và layo
ut câú hình UI cho từng route)
 ├── components/
 │   ├── layout/           # Sidebar, Header, PageContaine
r...
 │   ├── shared/           # DataTables, Modal, SearchBox (Dù
ng chung)
 │   └── ui/               # Nút bấm, Inputs, thẻ trạng thá
i... (Dumb components)
 ├── config/               # Chứa các config như Menu list, N
av items...
 ├── contexts/             # AuthContext...
 ├── hooks/                # Custom hooks (e.g., useDebounce,
useCenterSearch)
 ├── lib/
 │   ├── firebase/         # Config hiện tại
 │   └── utils/            # Các file helper
 ├── services/             # [MỚI] Data Access Layer - Nơi MỘ
```

```
T MÌNH NÓ được gọi Firestore
 └── types/                # [MỚI] Định nghĩa TypeScript Use
r, Center, TherapySession...
```

Tóm lại: Khởi đầu của bạn rất tốt (sử dụng được các công cụ hot nhất), nhưng project hiện đang là "Prototype" chứ chưa phải "Product". Bạn cần gấp rút tiến hành Refactor, tách mỏng logic và chia các component theo kiểu Lego (Atomic Design) trước khi nhồi nhét thêm tính năng mới! Cần mình hỗ trợ refactor file nào trước tiên không?